P3085R1

`noexcept` policy for SD-9 (throws nothing)

Published Proposal, 2024-03-17

This version:
https://wg21.link/P3085

Author:
Ben Craig

Audience:
LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Source:
github.com/ben-craig/freestanding_proposal/blob/master/library/throws_nothing_policy.bs

---

Abstract

Table of Contents

§ 1. Revision history

§ 1.1. R1

- Added principled design and notable omissions sections.
- Removed reference to undefined term "pass-through" function.

§ 1.2. R0

First revision!

§ 2. Introduction

The first priority of this paper is to establish a lasting policy on the usage of `noexcept` in the standard library that avoids re-litigation for a few years.

The second priority is to establish the author's preferred policy.

This paper currently recommends the policy that functions that LEWG and LWG agree cannot throw should be marked unconditionally `noexcept`.

This paper discusses an alternative policy that library functions that could exhibit undefined behavior due to precondition violations should not be marked `noexcept` (AKA the "Lakos rule", AKA the narrow `noexcept` policy).

With luck, one of these two policies can gain consensus.

§ 3. Prior `noexcept` Discussions

§ 3.1. History papers

- N2855, Rvalue References and Exception Safety, [GregorN2855]
- N2983, Allowing Move Constructors to Throw, [AbrahamsN2983]
- N3248, noexcept Prevents Library Validation., [MeredithN3248]
- N3279, Conservative use of noexcept in the Library, [MeredithN3279]
- P0884R0, Extending the noexcept Policy, Rev0, [Josuttis0884]
- P2920R0, Library Evolution Leadership's Understanding of the Noexcept Policy History, [Lelbach2920]

§ 3.2. Throws Nothing Rule

- P1656R2, "Throws: Nothing" should be noexcept, [Bergé1656]
- P2148R0, Library Evolution Design Guidelines, [Johnson2148]

§ 3.3. Narrow `noexcept` / Lakos Rule

- P2831R0, Functions having a narrow contract should not be noexcept, [Doumler2831]
- P2837R0, Planning to Revisit the Lakos Rule, [Meredith2837]
- P2861R0, The Lakos Rule: Narrow Contracts And noexcept Are Inherently Incompatible, [Lakos2861]
- P2949R0, Slides for P2861R0: Narrow Contracts and noexcept are Inherently Incompatable, [Lakos2949]
- P2946R0, A flexible solution to the problems of noexcept, [Halpern2946]

§ 3.4. Both

- P2858R0, Noexcept vs contract violations, [Krzemieński2858]
- This paper, P3085, `noexcept` policy for SD-9 (throws nothing)
- P3005R0, Memorializing Principled-Design Policies for WG21, [Lakos3005]

4. Policies in question

These two policies only differ on point b.

4.1. Common parts (proposed)

This wording matches the shared wording from [Bergé1656] and [Josuttis0884].

Instructions to the editor: Please add the following to the "List of Standard Library Policies" in SD-9.

4.1.1. `noexcept` policy {#wording_heading}

Rationale reference: P3085

a. No library destructor should throw. They shall use the implicitly supplied (non-throwing) exception specification.

b. *EDITOR'S NOTE: Fill in this option with the selected policy. See below.*

c. If a library swap function, move-constructor, or move-assignment operator is conditionally-wide (i.e. can be proven to not throw by applying the `noexcept` operator) then it should be marked as conditionally `noexcept`.

d. If a library type has wrapping semantics to transparently provide the same behavior as the underlying type, then default constructor, copy constructor, and copy-assignment operator should be marked as conditionally `noexcept` the underlying exception specification still holds.

e. No other function should use a conditional `noexcept` specification.

f. Library functions designed for compatibility with "C" code (such as the atomics facility), may be marked as unconditionally `noexcept`.

4.2. Throws Nothing Rule (proposed)

This wording does not match [Bergé1656], but it has the same intent.

b. Each library function that will not throw when called with all preconditions satisfied should be marked as unconditionally `noexcept`.

The wording in [Bergé1656] is as follows:

> b. Each library function that LEWG and LWG agree cannot throw, should be marked as unconditionally `noexcept`.

The [Bergé1656] wording could be interpreted as disallowing narrow contract functions, since a function that is called without meeting preconditions could do anything, including throw. That interpretation isn't the intent of [Bergé1656].

§ 4.3. Narrow `noexcept` / Lakos Rule (not proposed in this revision)

This wording matches the wording in [Josuttis0884].

> b. Each library function having a *wide* contract (i.e., does not specify undefined behavior due to a precondition) that the LWG agree cannot throw, should be marked as unconditionally `noexcept`.

§ 5. Status Quo

Currently, there is no policy on `noexcept`. Between 2011 and 2020, `noexcept` was applied to the standard library in line with the narrow `noexcept` rule [MeredithN3279] [Josuttis0884]. In 2020, the throws nothing rule was proposed [Bergé1656]. The designs that LEWG has forwarded to LWG between 2020 and the present day have been following the narrow `noexcept` rule.

There are 78 occurrences of "Throws: Nothing." in the standard [Cpp2023-12], but this number is misleading as many of the usages are in blanket wording that gets applied to multiple classes. The Microsoft Visual Studio 2022 Preview implementation (version 14.38.32919) annotates each `noexcept` they strengthen with a `/* strengthened */` comment. There are 633 hits of the "noexcept /* strengthened */" string in the headers for this implementation. All of these are either definitions of functions, or macros that make stamping out `<cmath>` overload definitions easier. In other words, these hits aren't counting both forward declarations plus definitions, they are only counting definitions. This search omits any conditionally `noexcept` functions.

There are roughly 3,600 instances of the string "noexcept;" in the standard. A few of these are in examples. For most `noexcept` functions, "noexcept;" will show up twice in the standard [Cpp2023-12], once in a synopsis and once in the detailed specification. This means there are approximately 1,800 unconditionally `noexcept` functions in the standard. This search omits any conditionally `noexcept` functions.

§ 6. Why should we establish any `noexcept` policy?

§ 6.1. Core-language Contracts

Interactions between `noexcept` and the upcoming core-language contracts facilities is likely to be significant. It would be unfortunate if decisions in the library prevented the core-language contracts work from reaching its full potential.

[Meredith2837] argues that the interaction may be significant and unpredictable enough that it would be better to proceed with no policy in the short term, rather than settle on a policy that will immediately get in the way of contracts.

If we don't set a policy, then we leave it to paper authors and LEWG to determine the proper `noexcept` specifier on a per-function basis. This is likely to cause a great deal of inconsistency in the interim.

§ 6.2. Consistency over preference

There are some design areas where the difference in two choices is of little consequence. LEWG will often spend some time rediscovering the trade-offs and existing practice around the differences, and then make a somewhat arbitrary decision. In these cases, LEWG tends to bias towards "whatever the bulk of the standard is currently doing".

LEWG could become more self consistent by documenting one way, and then requiring authors to either follow the policy, or provide rationale for why the policy should not be applied in this case.

The explicit-ness of multi-argument constructors is one such area [Voutilainen2711].

§ 6.3. Changes in direction

Sometimes, LEWG follows one design pattern in the standard, then discovers or invents a new pattern. The new pattern is inconsistent with the old pattern. Authors then wonder whether to follow the new pattern or old pattern.

This is a reasonable place for LEWG to document a policy.

Using hidden `friend`s for non-member operators is one example of a change in direction. Whether to make all new "free functions" in the standard library customization point objects (like `std::ranges`) is another.

§ 6.4. Cross group trust

There are a lot of discussions happening concurrently within WG21. Sometimes, one group wants to rely on conventions from another group. These policies help people focus on the groups they are needed, rather than spend a large portion of their time ensuring their policy is followed (or not) in LEWG.

An example policy here would be ensuring that `const` means either shallow bitwise const or internally synchronized. The concurrency study group is likely relying on that convention to be followed, so changing `const` directions would cause surprise.

§ 7. Why the LEWG `noexcept` policy isn't actually that important

Implementations already strengthen the `noexcept` status of many STL functions. MSVC STL, libc++, and libstdc++ all mark `vector::back()`, `vector::operator[]`, and many other functions as `noexcept`.

Marking existing functions in the standard `noexcept` will change the results of very few instances of the `noexcept` operator in practice.

If the committee continues to permit implementations to strengthen the `noexcept` status of functions, then implementations won't change.

If the committee chooses to remove this permission, implementations will either ignore the requirement and be non-conforming, or they will accept the code breakage that the backwards compatibility arguments § 8.2 Narrow to wide substitutability / backwards compatibility are trying to avoid. If we are breaking compatibility in this way once, we may very well break it again in the future. The main way changing the `noexcept` policy changes things for standardization and users is if we remove the permission to strengthen `noexcept` and implementations grudgingly take a one time (but not an N time) compatibility hit.

Note that this paper makes no attempt to change the `noexcept` strengthening rules in either it's proposed rules or the presented alternative.

§ 8. Arguments

This section is organized by topic, and not by policy. That means that sections discuss one topic at a time (e.g. testing), and then discuss the affect of the policies on the topic.

§ 8.1. Algorithmic optimizations and exception safety

[GregorN2855], [AbrahamsN2983], [MeredithN3248]

The original, primary motivation for `noexcept` was to make it possible for `vector` and similar constructs to take advantage of `move` optimizations, without breaking existing code (e.g. `vector`'s strong exception guarantee).

This motivation and use case is uncontested in the various papers debating the usage of `noexcept`, so this paper will not spend further time on it.

On most implementations, disabling exceptions does not change the result of the `noexcept` operator (The discontinued Intel ICC makes the `noexcept` operator return `true` unconditionally under `-fno-exceptions`). This means that `noexcept` is still needed in order to opt-in to algorithmic optimizations, even when exceptions are disabled.

§ 8.2. Narrow to wide substitutability / backwards compatibility

[Doumler2831], [Lakos2949], [Lakos2861]

Suppose we have a function `void unchecked_f(int x)` with a precondition of `x > 0`. `unchecked_f` has undefined behavior when the precondition is violated.

Now suppose we have a function `void checked_f(int x)` that has the same behavior as `unchecked_f` when the precondition is met, but it throws an exception when the precondition is violated. `checked_f` and `unchecked_f` are substitutable for each other, as all in-contract uses behave the same.

Now suppose we have a function `void noexcept_f(int x) noexcept` that has the same behavior as `unchecked_f` when the precondition is met. `noexcept_f` and `unchecked_f` aren't substitutable with each other, due to the change in behavior of the `noexcept` operator. `noexcept(noexcept_f(0))` returns `true`, and `noexcept(unchecked_f(0))` returns `false`. This change in behavior constrains implementations and the C++ committee.

In theory, avoiding `noexcept` gives WG21 more latitude to make changes in the future. In practice, this latitude is unneeded, and difficult to use.

The author is unaware of any previous proposal that attempted to make a "Throws: nothing" function released in one standard into a function that could throw something in a later standard. This suggests that this freedom to change isn't that valuable in practice. We've had the freedom, but haven't needed it or used it.

Using the freedom to make an out-of-contract call throw an exception would be a compatibility break in practice. MSVC STL, libc++, and libstdc++ all mark `vector::back()`, `vector::operator[]`, and many other functions as `noexcept`. If we were to require `vector::back()` to be `noexcept(false)` so that it could throw when the container is empty, it would cause a (minor) break for the majority of C++ code bases migrating to that standard due to the change in the `noexcept` operator's result. Perhaps such a break would be small enough to gain consensus in WG21 though.

Major implementations don't currently use `noexcept` latitude to implement throwing precondition checks. The three major implementations (MSVC STL, libstdc++, and libc++) all have checked implementations. All three use some variation of termination as their mechanism of handling a failed check.

Less prevalent implementations do use such latitude for checked implementations and library testing.

8.3. Codegen changes

8.3.1. "Modern" implementations

Most platforms use the "table-based" implementation strategy for exceptions. The combinations of (x86-64, ARM, Aarch64) X (Windows, Linux, iOS, OSX) all use table-based exceptions.

With table-based implementations, `noexcept` changes the generated assembly. There are sometimes extra stack manipulations, and minor bookkeeping changes, but no extra branches. Those minor changes affect the performance in a small and measurable way, but sometimes that difference is positive (for `noexcept`), and sometimes negative (against `noexcept`). The magnitude of the difference is under six cycles for the programs and compilers tested [Craig1886]. Full application testing might show some instruction cache effects, but it is even more likely that any differences would be lost in the noise and immeasurable.

8.3.2. Other implementations

[Doumler2831]

Not all implementations use the table-based implementation strategy. These implementations have more substantial bookkeeping relative to table-based implementations.

The most well known of these implementations is the Microsoft Windows 32-bit x86 implementation. Each new cleanup context requires manipulating a linked list of cleanup actions, even on the happy path. This results in substantial extra code, and more than a 20% execution performance penalty with micro-benchmarks relative to exceptions being disabled [Craig1886]. With heavy use of `noexcept`, the overhead can be removed. The default and recommended exception project settings of the Microsoft

Visual Studio compilers is `/EHsc`, which (non-conformingly) makes `extern "C"` functions `noexcept` by default as a way to reduce exception overhead.

While the days of peak Microsoft Windows for 32-bit x86 are far behind us, new compilers targeting the platform are still regularly released and downloaded. Microsoft Visual Studio 2022 (the latest version at time of writing) still ships compilers targeting 32-bit Windows. Clang 17.0.5 (released on Nov 14, 2023) still targets 32-bit Windows. Download statistics for Clang binaries are available through the GitHub API.

```
// Clang 17.0.5 download statistices, gathered on Feb 2, 2024
// https://api.github.com/repos/llvm/llvm-project/releases
LLVM-17.0.5-win32.exe : 1422 downloads
LLVM-17.0.5-win64.exe : 22370 downloads
LLVM-17.0.5-woa64.exe : 304 downloads
clang+llvm-17.0.5-aarch64-linux-gnu.tar.xz : 532 downloads
clang+llvm-17.0.5-arm64-apple-darwin22.0.tar.xz : 585 downloads
clang+llvm-17.0.5-x86_64…gnu-ubuntu-22.04.tar.xz : 1803 downloads
clang+llvm-17.0.5-powerpc64-ibm-aix-7.2.tar.xz : 69 downloads
```

Readers should not try to make too many inferences about the relative popularity of platforms based on these numbers, as developers can get compiler releases from multiple sources that wouldn't register on the GitHub download page. Regardless, the Microsoft Windows 32-bit platform still gets used, even with the most recent compilers, so it should not be brushed off as irrelevant.

James Renwick has an alternative implementation of exceptions [Renwick2019] with the intent of improving determinism in embedded systems. This implementation also needs to manipulate bookkeeping information in each new cleanup context. The bookkeeping information is passed along to potentially throwing functions through a hidden parameter. `noexcept` can remove the cost of the hidden parameter, but can introduce a new cleanup context.

On GPUs, there is a large cost to having additional control flow edges. Using `noexcept` can remove control flow edges. However, to the author's knowledge, no GPU currently supports C++ exceptions.

§ 8.3.3. Bloat

[Bergé1656], [Doumler2831], [Lakos2861]

The "tables" backing table-based exception handling, and the code implementing non-table-based bookkeeping all consume space. Adding `noexcept` can reduce code bloat, as this can reduce the amount of information that needs to go into tables or bookkeeping. Adding `noexcept` can also cause

bloat if the `noexcept` function calls potentially throwing functions, as this transformation may need to introduce table entries or bookkeeping to ensure that `std::terminate` gets called.

In Compiler Explorer, bloat is best examined by disabling debug information (`-g0` for Clang and GCC), enabling optimizations (`-O2`), and leaving directives in the output (uncheck the box in "Filter..."). As a (very rough) estimate of bloat, we can use lines of assembly output as a proxy. Some of the lines don't end up as bytes in the resulting binary, but many do show up, often as variable length integers. Measuring the results on actual binaries is substantially more difficult, due to section alignment quantizing and obfuscating size differences. [Craig1640] presents size benchmarks comparing exceptions to abort, and some of these measurements can be used as proxy byte-based statistics for `noexcept` bloat.

This code makes it possible to see what happens to the quantity of assembly when switching whether `caller` is `noexcept` or not, and whether `callee` is `noexcept` or not. All measurements are in the very rough lines of assembly measurement.

Compiler	**`noexcept(false)` calls `noexcept(false)`**	**`noexcept(false)` calls `noexcept(true)`**	**`noexcept(true)` calls `noexcept(false)`**	**`noexcept(true)` calls `noexcept(true)`**
x86 msvc v19.38	76	25	70	25
x64 msvc v19.38	44	26	37	26
x86-64 gcc 13.2	85	20	32	20
x86-64 clang 17.0.1	68	22	76	22
ARM64 gcc 13.2.0	69	26	46	26
armv8-a clang 17.0.1	74	30	88	30

For many platforms and applications, this bloat is inconsequential. For many platforms and applications that are size constrained, exceptions are already disabled. When exceptions are disabled, the "`noexcept(true)` calls `noexcept(true)`" code and the exception disabled code take the same number of lines of assembly.

§ 8.4. Library Testing

[MeredithN3248], [Doumler2831]

Implementations often constrain some undefined behavior for out-of-contract function invocations, particularly in debug and checked modes. "Negative testing" validates that the assertion / contract checks are authored correctly, and that the resulting constrained undefined behavior does the expected thing. The contract checks are code, and code needs testing.

Negative tests can be authored for codebases using the narrow `noexcept` rule or the throws nothing `noexcept` rule, but it is substantially easier to do so for narrow `noexcept` rule codebases.

§ 8.4.1. Core-language contracts

Usage of the proposed core-language contracts facilities will need to be tested. `noexcept` will present testing challenges when violations of core-language contracts are configured to throw an exception. Death tests are an option, but they have a large set of challenges on their own.

§ *8.4.1.1. Put the contract check on the outside of the `noexcept`*

One of the design decisions that core-language contracts needs to make is to decide exactly where the checks will be run. If precondition and post-condition checks happen within the callee (i.e. inside the `noexcept`), then failed preconditions and post-conditions checks set to throw will cause the program to terminate. If those checks happen in the caller of the function (i.e. outside the `noexcept`), then at least one layer of throwing can still work in testing. Deeply nested checks can still cause a terminate if they end up propagating through a different `noexcept`. If precondition and post-condition checks in the caller of a `noexcept` function, then that would resolve most of the core-language contracts negative testing issues.

Putting preconditions and post-conditions outside the `noexcept` doesn't help with core-language contract asserts.

[Voutilainen2780] discusses this point, as well as discussing other benefits involving calling external libraries. It also discusses the challenges of this approach with regards to indirect calls and multiply evaluated preconditions.

Caller-side precondition checking is likely to be less efficient in terms of code size compared to callee-side precondition checking.

8.4.1.2. Retrieve precondition and post-condition results via reflection

Suppose we have the following function + contract:

```
int f(int x) noexcept
  pre (x >= 0)
  post(r : r % 2 == 0);
```

Now imagine we had reflection facilities that let us do something like the following:

```
// // evaluate contracts of f(x) without calling f(x)
EXPECT_TRUE(  $evaluate_pre(f, 20) );
EXPECT_FALSE( $evaluate_pre(f, -1) );
EXPECT_TRUE(  $evaluate_post(f, 2, 20) ); // x = 20, returns 2
EXPECT_FALSE( $evaluate_post(f, 1, 0) ); // x = 0, returns 1
```

This would allow directly testing contracts without violating those contracts. The tests would be very fast (no throws or terminates involved), and low maintenance.

To the author's knowledge, no such facility is currently proposed. This reflection utility has applications beyond testing, such as automatically generating "wide" wrapper functions from "narrow" functions. This strawman proposal doesn't address core-language asserts.

8.4.1.3. Allow contract build modes to change the behavior of `noexcept`

Perhaps we consider `noexcept` the first core-language contract, with a default violation handler of `std::terminate`. The `Eval_and_throw` mode could change the behavior of `noexcept` to observe exceptions instead of `terminate`.

This may even be something that should be extended to a general sad-path post-condition checking facility. It would be nice if we could express that, on failure, `std::vector`'s single element insert should leave the container with the same `size()`, and the same `data()` pointer. Making such sad-path post-condition checks throw on failure would also lead to early termination.

Allowing contract build modes to change the behavior of `noexcept` could be a concise way to unify `noexcept` and core-language contracts, but it could come at a substantial compatibility cost for those using core-language contracts. If their code was relying on `noexcept` terminating, then enabling `Eval_and_throw` will break their program.

This approach may be able to be combined with § 8.4.7.2 Exploiting precondition undefined behavior to ignore termination to avoid major compatibility breaks.

§ 8.4.2. Extract contract predicates to a function

A user could emulate the precondition reflection by following a good naming convention.

```
bool precondition_f(int x) noexcept;
bool postcondition_f(int retval, int x) noexcept;
int f(int x) noexcept {
  assert(precondition_f(x));
  int retval = 0;
  /*...*/
  assert(postcondition_f(retval, x));
  return retval;
}

EXPECT_TRUE(  precondition_f(20) );
EXPECT_FALSE( precondition_f(-1) );
EXPECT_TRUE(  postcondition_f(2, 20) );
EXPECT_FALSE( postcondition_f(1, 0) );
```

This is an approach that is available for today's precondition and post-condition checking facilities. A sophisticated user could write static analysis checks to enforce such a convention. The author is unaware of any libraries or guidelines that do this though, which suggests that users either didn't think of it, or they don't see this approach as a good return on investment.

§ 8.4.3. The widely deployed implementations don't diagnose contract violations with exceptions

[Bergé1656], [Doumler2831]

In 2023, the three most widely deployed standard libraries are libc++ (shipping with Clang), libstdc++ (shipping with GCC), and Microsoft's Visual Studio standard library (shipping with Microsoft Visual Studio). None of these implementations use exceptions to diagnose contract violations.

§ 8.4.4. `noexcept` macro

[MeredithN3248], [Doumler2831]

Libraries can define a macro like the following:

```
#if TEST_ASSERTIONS
  #define MY_NOEXCEPT
#else
  #define MY_NOEXCEPT noexcept
#endif
```

This approach works, but it requires a large number of annotations. Switching between the modes is detectable and increases the number of differences between the test environment and production environment. Each difference between test and production decreases the value of the test.

§ 8.4.5. setjmp/longjmp

[MeredithN3248], [Doumler2831]

`setjmp` and `longjmp` can be used to bypass `noexcept` without triggering termination. It is very difficult to use `setjmp` and `longjmp` without triggering undefined behavior though. That makes `setjmp` and `longjmp` generally not viable.

§ 8.4.6. Child threads, fibers, and signals

[Doumler2831]

There are other creative ways to write contract handlers that permit testing while avoiding having an exception run into a `noexcept`. [Doumler2831] describes some approaches for using (and leaking) threads that halt on failure rather than return. There's also a description for how to use fibers / stackful coroutines to halt without consuming as many resources as the child thread approach. Finally there's a signal based approach. All of these approaches have substantial downsides. None of these approaches are in wide use to the best of the author's knowledge.

§ 8.4.7. Potentially mitigating compiler features

There is the potential to address some of the testability concerns of `noexcept` with compiler extensions.

These ideas aren't new. Some of them have been floating around for more than 10 years.

The fact that they haven't been implemented says something. That something may be "Upstreaming compiler changes is an expensive prospect, and outside the capabilities of most organizations." It may be "I'll do negative testing in a way that doesn't require a compiler feature." Or it may be "Negative testing isn't sufficiently valuable to jump through these hoops." So I'm not entirely sure what the lack of implementations says, but it certainly says something.

8.4.7.1. Compiler flag that makes `noexcept` unchecked

The `noexcept` specifier has two direct effects on the semantics of a program:

- exceptions leaving a `noexcept` function trigger `std::terminate`, and
- uses of the `noexcept` operator on expressions with the `noexcept` specifier change.

In theory, a compiler could add a non-conforming extensions that removes the `std::terminate` aspects, while keeping the `noexcept` operator aspects. This would make it easier to test contracts, as then any exceptions could still escape `noexcept` functions.

In some ways, this is the inverse of [Halpern2946]. [Halpern2946] proposes a new attribute that (conditionally) keeps the `std::terminate` semantics, but doesn't modify the `noexcept` operator's behavior.

8.4.7.2. Exploiting precondition undefined behavior to ignore termination

If a function does not meet its preconditions, then it is not required to meet its post-conditions. A post-condition of `noexcept` functions is terminating when there's an exception. An implementation could, conformingly, bypass the termination aspect of `noexcept` if the program has encountered undefined behavior. In practice, this may be done by having a "magic" exception type (e.g. `nonstd::undefined_behavior_exception`) that bypasses `noexcept`. Users would then be required to only use that exception when undefined behavior has been encountered.

8.4.8. Death tests

[Bergé1656], [Doumler2831]

A commonly recommended approach for negative testing is to use "Death tests". With death testing, the code under test is run in a different process. When the code under test executes a failing contract check, the code under test is expected to exit abnormally. The exit value of the child process is checked in the

parent process to ensure that the contract check was triggered appropriately. Multiple standard libraries use death tests for their negative testing.

The largest complaint regarding death testing is the performance penalty. Launching a process is orders of magnitude more expensive than throwing an exception, and this can inflate test times from seconds to minutes when large numbers of negative tests are present. The performance penalty is particularly large on Microsoft Windows.

Traditionally, death tests can communicate a very limited amount of information from the child process to the parent process, typically one integer or less. This makes it difficult to know whether the child process's abnormal termination was from the intended contract check failure, or some other reason. In theory, the failing checks could communicate through some other out-of-band channel (a file on disk, standard out / error, shared memory), but this isn't currently common practice.

Launching child processes has other hazards as well. Using the POSIX `fork()` call while code under test is running in another thread introduces substantial testing risks and uncertainty. If the code under test is not `fork()` safe, then death tests can't be launched with `fork()` while the code under test is running. This can add yet-another performance penalty, as it forces the test to re-run potentially expensive setup code. The test framework and code under test also have to take care not to run `fork()` unsafe code, which can be challenging when global constructors are involved. A mitigation for this is to have a test launcher that is entirely distinct from the code under test. `spawn()` and `clone()` calls can also be used, but with their own draw backs.

Death tests have portability issues. C++ can run in a variety of environments, and death tests are mostly suited to desktop and server environments. Implementing death tests in browsers and embedded / microcontroller environments would be very challenging, as they typically don't support multiple processes. In addition, most testing frameworks don't support death tests, with GoogleTest being the main test framework with that support. GoogleTest mixes the code under test and the test infrastructure in the same process(es).

§ 8.5. Using exceptions to diagnose precondition violations in production

§ 8.5.1. Security dangers of throwing exceptions while in an unknown state

In security sensitive contexts, running out of contract is very dangerous. Each instruction executed while out of contract is another opportunity for an attacker to do as they please with the system. Programs should run as little code as possible after detecting contract violations. This is typically done by exiting the program as quickly as possible.

Throwing an exception runs a lot of instructions. Many of those instructions are indirect code branches, which are particularly dangerous. The Microsoft Visual Studio 32-bit x86 implementation has a build flag (`/SAFESEH`) to harden their exception handling implementation against attacks attempting to leverage those indirect code branches, but the number of instructions run is still very high.

§ 8.5.2. Temporary continuation

[Lakos2861]

Following the narrow `noexcept` policy is more permissive in terms of allowing temporary continuation than the throws nothing policy. Some users of C++ desire temporary continuation after contract violations, and the throws nothing policy prevents temporary continuation in many places.

§ 8.5.3. Likely to cause more UB or hit a destructor `noexcept`

Writing exception safe code often requires using operations that aren't allowed to throw. A common pattern is to "do all the work off to the side, then commit using nonthrowing operations only" [Sutter82]. If one of the non-throwing operations ends up throwing due to a contract violation, then additional library UB has been added to the program, beyond the initial contract violation.

Throwing from a non-throwing operation can also cause termination in destructors, which are `noexcept` by default. So if a contract is violated in a throws nothing function called from a destructor, then termination is the behavior, even if continuation is the desire.

Both of these points illustrate the difficulty and danger in requiring a program to never terminate, even when aided by throwing contract handlers.

§ 8.5.4. Postconditions only hold when preconditions hold

[Krzemieński2858]

A general programming principle is that postconditions aren't required to hold when preconditions don't hold. This is often paraphrased as garbage in, garbage out. If `noexcept` is a postcondition, then the termination aspect is not required to hold if the precondition doesn't hold.

Here is an example program where the violation of a library precondition leads to language undefined behavior, with an end result of `noexcept` being bypassed by an exception.

§ 8.5.5. Precondition UB and `noexcept` contradict?

[Doumler2831]

[Doumler2831] argues that `noexcept` and precondition undefined behavior contradict, as any behavior should be permitted. `noexcept` prevents the implementer of the function from manifesting some of those behaviors.

For user code, the extent of their undefined behavior is indeed constrained by `noexcept`. Standard library implementations are under no such restriction. § 8.4.7.2 Exploiting precondition undefined behavior to ignore termination describes one way that an implementation could choose to throw an exception past `noexcept`, if they deemed it worth the implementation effort.

§ 8.6. Termination risk

[Doumler2831], [Lakos2861], [Lakos2949]

Using `noexcept` can insert code paths that invoke `std::terminate`. The terminating code paths are often unreachable code removed during translation, but this isn't always the case.

There are applications that prefer library UB over `std::terminate` when faced with a failing contract. Those libraries may use exceptions to indicate failed contracts.

For safety critical systems (note the use of the term "systems", not applications), undefined behavior is generally considered worse than termination, even if the undefined behavior is "only" library UB. In these systems, an individual application may terminate, but the system as a whole has ways to recover from a terminated application, or to put the system into a safe state.

§ 8.7. Philosophy

§ 8.7.1. `noexcept`-accuracy

[Bergé1656], [Doumler2831], [Halpern2946]

There is a desire to document the exception behavior of a function in the signature of that function. This documentation helps compilers, static analyzers, and people reading the code to understand what the function does.

[Krzemieński2858] makes the distinction between functions that can't throw and functions that can't fail. C++'s `fclose` can't throw (POSIX's can as part of thread cancellation). `fclose` can fail though,

as part of `fclose` is flushing buffer contents to storage, and that storage may not be available at time of close (imagine a USB drive being removed).

This separation between can't throw and can't fail makes it more difficult to determine what constitutes `noexcept`-accuracy.

§ 8.7.2. Doesn't matter since implementations can already add `noexcept`

[Doumler2831], [Bergé1656]

If the narrow `noexcept` policy is in place, implementations can strengthen the `noexcept` annotations on their functions. Major implementations have done so. Strengthening `noexcept` in the standard would end up adding very few `noexcept` annotations in the major implementations.

§ 8.7.3. Standard library policies vs. C++ library policies

[Lakos2861], [Doumler2831]

Many libraries attempt to emulate the design policies of the standard library. The standard library should try to set a good example for third-party and end-user libraries, as others will use the standard library as an example whether the committee thinks they should or not.

The standard library needs to accommodate all domains, serve millions of programmers, and billions of users. Most other libraries aren't as widely used. In some libraries, the customers are known well enough that a compatibility hit from changing `noexcept` can be absorbed. The domain and use cases are known well enough that termination on precondition violation can be deemed suitable.

For some libraries, improving some performance aspect (like binary size) by half a percent is worth increasing the cost of testing by a factor of 100. The more heavily used a library, the more likely it is that such a trade-off is worthwhile. Few libraries are used more heavily than the STL.

§ 8.7.4. Standard library implementation strategies need not be encoded in the C++ standard

[Lakos2861]

For some standard library implementations, the correct technical and business choice is to terminate on precondition failure via a `noexcept` decoration. That is a valid implementation strategy. However, that doesn't mean that this implementation should be put in the standard, where it constrains other implementations that don't want to make that choice.

§ 8.7.5. Cleanup operations often have preconditions, and need to not throw

[Krzemieński2858]

Cleanup operations, like `delete`, `fclose`, and `std::lock_guard::~lock_guard` all have preconditions, and all need to be made to not throw. If the preconditions on these functions were checked and made to throw, they would cause the `noexcept` on destructors to terminate. If the destructors were marked `noexcept(false)`, you would still run the risk of termination from when unwinding triggered a precondition fault, causing two exceptions to be active at the same time.

§ 8.7.6. Narrow `noexcept` ignores implicit precondition of indeterminate values and invalid objects

[Lakos2949]

The narrow `noexcept` policy is defined in terms of wide and narrow contracts, but even the wide contracts have their limits. Take `std::string::c_str()` as an example. `c_str()` is safe to call on any valid `std::string`, even the empty string. However, if the bytes composing the `std::string` are corrupted in some way, then `c_str()` is likely to dereference an invalid pointer. This means that `c_str()` (and most functions) have a precondition of basic object validity. If wide contracts are expanded to encompass functions that work with invalid objects, then very little has a wide contract.

§ 8.8. Can't write narrow `noexcept` on top of throws nothing

[Meredith2837]

If part of a system is written with a throws nothing policy, then the system as a whole loses the narrow `noexcept` property.

Systems built with libc++, libstdc++, and the Microsoft Visual Studio standard library generally do not have the narrow `noexcept` property.

Other libraries can get many of the benefits of narrow `noexcept`, even if the system as a whole doesn't have the narrow `noexcept` property. Negative testing can be performed on narrow `noexcept` libraries, as exceptions usually don't need to travel far. Preconditions can be checked in production for all the code between entry points and the first non-narrow `noexcept` code. Non-narrow `noexcept` libraries that interact heavily with callbacks can get in the way of both precondition checking and negative testing.

§ 9. Notable Omissions

This paper's discussions focus almost entirely on rule "b" in the `noexcept` policy. The rationale for the other rules haven't been explored nearly as much. Given that, I will list some known issues that should be considered unaddressed, should someone decide to challenge the other rules.

§ 9.1. Asynchronous callbacks

[StdExec2300] uses the `noexcept` operator in conjunction with concepts to make compile-time decisions. The proposed policies do not place any requirements on concepts or user facilities, but the policies do place restrictions on functions. Many [StdExec2300] functions are marked `noexcept`, but it is unclear to the author of this paper whether any of those functions have preconditions.

This paper does not attempt to craft policy regarding `noexcept` and asynchronous callbacks.

§ 9.2. Throwing destructors

Destructors that `throw` exceptions are uncommon, and usually a bad idea. However, there are some categories of objects where doing so is useful, like `scope_success` in the library fundamentals v3 TS [LFTS4948].

This paper does not attempt to craft policy that distinguishes when having a throwing destructor is acceptable.

§ 9.3. Pervasive conditional `noexcept`

Some library authors have expressed desires to standardize conditional `noexcept` operations for facilities like duration math in `chrono`. `chrono::duration` instantiations involving built in integers have many non-throwing wide contract operations that become throwing when using "BigNum" types.

The proposed policies prohibit standardizing conditional `noexcept` for functions other than some special member functions. This paper provides no rationale for this choice.

§ 9.4. C compatibility

The proposed policy permits C compatibility facilities to be marked unconditionally noexcept. This paper provides no rationale for this choice.

§ 10. Principled design

[Lakos3004] describes a process of enumerating, ranking, and scoring design principles as a way to aid decision making and communication. The associated paper [Lakos3005] includes a worked version discussing `noexcept` policy variations. This paper will present an alternative compliance table, as well as discuss some of the big differences between the [Lakos3005] table, and the one presented here.

§ 10.1. Principle descriptions

§ 10.1.1. Shared principles

The following principles are shared between [[Lakos3005] and this paper, though the wording may be slightly modified.

- AlgoOpt: Maximize algorithmic runtime optimizations. § 8.1 Algorithmic optimizations and exception safety
- MinExeSize: Minimize executable and shared library size. § 8.3.3 Bloat
- MinObjSize: Minimize object-code size. § 8.3.3 Bloat
- MaxExpressInCode: Maximize expressing defined behavior directly in code and the standard. § 8.7.1 noexcept-accuracy
- MinUnintendedExit: Minimize the chance that a propagated exception will cause unintended termination. § 8.6 Termination risk
- MaxNegUnitTest: Maximize the return on investment for negative unit testing of standard functions. § 8.4 Library Testing
- MinCostToImpls: Minimize effort for implementations to remain conformant while satisfying their client base.
- MinCostToFixStd: Minimize effort to align the current Standard Library with this policy statement.
- MinPermDiverge: Minimize likely persistent discrepancies between the Standard and this policy statement.
- MaxUserPortability: Maximize portability for user code written using the Standard Library.

§ 10.1.2. New principles

The following principles are new in this paper.

- TableBasedPerf: Maximize runtime performance on table based implementations. § 8.3.1 "Modern" implementations
- NonTablePerf: Maximize runtime performance on non-table based implementations. § 8.3.2 Other implementations
- MinUnsafe: Minimize utility of unsafe, insecure, and misleading patterns. § 8.5.1 Security dangers of throwing exceptions while in an unknown state, § 8.5.3 Likely to cause more UB or hit a destructor noexcept
- FutureWidening: Maximize committee's ability to widen function contracts in the future § 8.2 Narrow to wide substitutability / backwards compatibility
- MaxContractRecovery: Maximize user's ability to recover from a contract violation without terminating § 8.5.2 Temporary continuation, § 8.6 Termination risk

§ 10.1.3. Omitted principles

The following principles from [Lakos3005] are not scored or ranked in this paper.

- ArgRestrict: Maximize use of noexcept to qualify function parameters.
- AllowOnAsyncFrame: Allow noexcept on customization-point functions used in asynchronous contexts.

These principles seem to be related to questions involving asynchronous callbacks. This paper is choosing to not address asynchronous callback noexcept questions at this time. § 9.1 Asynchronous callbacks

- MaxUserSafeShut: Maximize implementations' ability to enable users to achieve safe shutdown in production.
- MaxUserSafeRecov: Maximize implementations' ability to enable users to handle and continuing in production.

These principles are replaced by MaxContractRecovery.

In addition, these principles conflate safety, shutting down cleanly, and recovery with throwing contract-violation handlers.

If future revisions of this paper are needed, then more detailed arguments about safe shutdown and safe continuation can be provided. § 8.5 Using exceptions to diagnose precondition violations in production contains some discussion.

- MaxImplFlexHand: Maximize implementations' ability to support throwing contract-violation handlers.

This principle is replaced by MaxContractRecovery.

MaxImplFlexHand isn't directly useful. If a contract-violation handler throws, and almost immediately terminates due to it hitting a `noexcept`, then that is operating as specified, and therefore easy to support. MaxContractRecovery attempts to capture the use case of contract failures being recoverable.

- MaxBestPracExamp: Maximize a desirable best-practices model (safety/performance) for typical retail users.

§ 8.7.3 Standard library policies vs. C++ library policies discusses the topic. Scoring this topic is challenging, as what constitutes the best practice is up for debate. This means that the scoring for such a policy is circular, in that we need to know the solution outcome to know a proper score for the principle. As a result, I chose not to score this policy.

- MaxUseForAlgoNeed: Maximize ability to use noexcept for narrow contracts having algorithmic need.

This seems like a duplicate of AlgoOpt "Maximize algorithmic runtime optimizations".

- MinDivergMeaning: Minimize divergence in meaning of standard functions across conforming implementations.

This seems to be a duplicate of MaxUserPortabilty "Maximize portability for user code written using the Standard Library". Divergence in behavior causes portability issues.

- MaxMultiverse: Minimize the likelihood of disenfranchising a member of the multiverse. § 8.8 Can't write narrow noexcept on top of throws nothing

This is more of a meta-principle. This meta-principle encourages prioritizing cutting solutions with low principle scores, even when a given solution may win out on more important principles.

§ 10.2. Solution descriptions

This paper describes two solutions, but represents it as three columns in the compliance table. The narrow `noexcept` solution is represented twice, once for the "minority" implementations that only use `noexcept` where mandated by the standard, and once for the "majority" implementations that place `noexcept` on most functions that throw nothing (as permitted by the standard). This will allow readers to gauge the impact in the different implementations.

- MinNar: Minority narrow `noexcept`. The minority implementations place `noexcept` only where required.

- MajNar: Majority narrow `noexcept`. These implementations place `noexcept` on most "throws nothing" functions. Closest to status quo.
- TN: Throws nothing rule

Most of the standard library is specified using the narrow `noexcept` design principle. It _almost_ counts as the status quo policy. However, we currently don't have a formal policy. This no-policy status quo is not represented in the table, as it is very difficult to reason about the properties of an "anything goes" approach. This is one of the reasons the author would prefer _a_ policy, even if it is not the author's preferred policy.

MinNar and MajNar correspond to solution C (MAX+LAK+NLC) in [Lakos3005]. TN corresponds to solution B (MAX) in [Lakos3005].

The author did not have the time to apply the principled design approach to the large variety of options explored in [Lakos3005]. There's only so much time between the February 2024 mailing and the 2024 March Tokyo meeting.

§ 10.3. Compliance table

Rank	Importance	Objectivity	Principle ID	MinNar	MajNar	TN
1	9	@	AlgoOpt	9	9	9
2	9	9	TableBasedPerf	9	9	9
3	5	9	NonTablePerf	5	9	9
4	5	9	MinExeSize	5	9	9
5	5	5	MaxExpressInCode	3	5	7
6	5	5	MaxUserPortability	7	7	9
7	5	5	MinUnintendedExit	7	5	5
8	5	5	MaxNegUnitTest	7	3	3
9	5	5	MinUnsafe	3	7	7
10	5	-	MinCostToImpls	9	9	7
11	1	5	FutureWidening	5	5	3
12	1	5	MaxContractRecovery	7	3	3
13	1	-	MinCostToFixStd	7	3	3
14	1	-	MinPermDiverge	7	3	3
15	-	9	MinObjSize	5	9	9

§ 10.4. Comparison with [Lakos3005]

Many of the scores for the solutions differed by small amounts in the papers. Small, 2 point changes won't be elaborated on here except where it moves a score away from 1.

[Lakos3005] places an importance of 9 on the MaxImplFlexHand, MaxUserSafeShut, and MaxUserSafeRecov policies. This paper gives MaxContractRecovery an importance of 1.

This paper places a higher importance on MinExeSize than [Lakos3005] (importance 5 vs. 1). Time and space efficiency have always been key concerns of C++, even in the Design & Evolution [StroustrupDE] days.

This paper scores MinUnintenedExit much higher for both policies. Very little standard library code that invokes user code would be marked `noexcept` with either policy, so the [Lakos3005] scores of 1 and 3 seem unwarranted. `noexcept` on destructors and `std::thread` are going to be very common causes of unintended exits, and their behaviors are the same in this paper and [Lakos3005].

MaxNegUnitTest is higher in importance here (5 vs. 1). The scores are also closer together. Negative testing is possible with the throws nothing policy, though they are more expensive. The return on investment for negative testing with narrow noexcept isn't ideal, as it requires using the very slow exception sad path.

§ 11. Suggested polls

§ 11.1. Either policy is better than no policy

> **Poll:** Having one of the two proposed `noexcept` policies in place is more important than having my preferred policy, so a head-to-head poll between the two presented alternatives is an acceptable way to select the policy.
>
> 5-way poll (SF/WF/N/WA/SA)

If the "any policy" poll gains consensus, then we do a head-to-head poll on the policies.

If the "any policy" poll does not gain consensus, then we do two 5 way polls. Whichever poll has consensus in general, and has more consensus than the other, wins. It is possible that neither has consensus. If LEWG is a little crazy, then both may have consensus.

If we do these as electronic polls (and we should), then all four polls should run.

§ 11.2. Head-to-head policy poll

> **Poll:** Which policy do we adopt?
>
> 3-way poll (narrow `noexcept`/"Lakos rule")/Neutral/"Throws nothing rule"

§ 11.3. Adopt the narrow `noexcept`/"Lakos rule" `noexcept` policy

> **Poll:** Add the narrow `noexcept`/"Lakos rule" `noexcept` policy to SD-9
>
> 5-way poll (SF/WF/N/WA/SA)

§ 11.4. Adopt the "Throws nothing rule" `noexcept` policy

> **Poll:** Add the "Throws nothing rule" `noexcept` policy to SD-9
>
> 5-way poll (SF/WF/N/WA/SA)

§ 12. Acknowledgments

Thanks to David Sankel for providing wording suggestions.

§ References

§ Informative References

[AbrahamsN2983]
David Abrahams; Rani Sharoni; Douglas Gregor. *N2983, Allowing Move Constructors to Throw*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2009/n2983.html

[Bergé1656]
Agustín Bergé. *P1656R2, "Throws: Nothing" should be noexcept*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1656r2.html

[Cpp2023-12]
Thomas Köppe. *N4971: Working Draft, Programming Languages -- C++*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4971.pdf

[Craig1640]

Ben Craig. *P1640R1, Error size benchmarking: Redux*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1640r1.html

[Craig1886]

Ben Craig. *P1886R0, Error speed benchmarking*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1886r0.html

[Doumler2831]

Timur Doumler; Ed Catmur. *P2831R0, Functions having a narrow contract should not be noexcept*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2831r0.pdf

[GregorN2855]

Douglas Gregor; David Abrahams. *N2855, Rvalue References and Exception Safety*. URL: https://www.open-std.org/JTC1/SC22/WG21/docs/papers/2009/n2855.html

[Halpern2946]

Pablo Halpern. *P2946R0, A flexible solution to the problems of noexcept*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2946r0.pdf

[Johnson2148]

CJ Johnson; Bryce Adelstein Lelbach. *P2148R0, Library Evolution Design Guidelines*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2148r0.pdf

[Josuttis0884]

Nicolai Josuttis. *P0884R0, Extending the noexcept Policy, Rev0*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0884r0.pdf

[Krzemieński2858]

Andrzej Krzemieński. *P2858R0, Noexcept vs contract violations*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2858r0.html

[Lakos2861]

John Lakos. *P2861R0, The Lakos Rule: Narrow Contracts And noexcept Are Inherently Incompatible*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2861r0.pdf

[Lakos2949]

John Lakos. *P2949R0, Slides for P2861R0: Narrow Contracts and noexcept are Inherently Incompatable*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2949r0.pdf

[Lakos3004]

John Lakos, Harold Bott, Mungo Gill, Lori Hughes, Alisdair Meredith, Bill Chapman, Mike Giroux, Oleg Subbotin. *P3004R0, Principled Design for WG21*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3004r0.pdf

[Lakos3005]

John Lakos, Joshua Berne, Harold Bott, Mungo Gill, Alisdair Meredith, Bill Chapman, Mike Giroux, Oleg Subbotin. *P3005R0, Memorializing Principled-Design Policies for WG21*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3005r0.pdf

[Lelbach2920]

Bryce Adelstein Lelbach; et al. *P2920R0, Library Evolution Leadership's Understanding of the Noexcept Policy History*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2920r0.pdf

[LFTS4948]

Thomas Köppe. *N4948: Working Draft, C++ Extensions for Library Fundamentals, Version 3*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4948.html

[Meredith2837]

Alisdair Meredith; Harold Bott Jr.. *P2837R0, Planning to Revisit the Lakos Rule*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2837r0.pdf

[MeredithN3248]

Alisdair Meredith; John Lakos. *N3248, noexcept Prevents Library Validation*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2011/n3248.pdf

[MeredithN3279]

Alisdair Meredith; John Lakos. *N3279, Conservative use of noexcept in the Library*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2011/n3279.pdf

[Renwick2019]

James Renwick; Tom Spink; Björn Franke. *Low-cost deterministic C++ exceptions for embedded systems*. URL: https://www.pure.ed.ac.uk/ws/portalfiles/portal/78829292/low_cost_deterministic_C_exceptions_for_embedded_systems.pdf

[StdExec2300]

Michał Dominiak; et al. *P2300R7: std::execution*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2300r7.html

[StroustrupDE]

Bjarne Stroustrup. *The Design and Evolution of C++*. URL: https://www.stroustrup.com/dne.html

[Sutter82]

Herb Sutter. *Exception Safety and Exception Specifications: Are They Worth It?*. URL: http://www.gotw.ca/gotw/082.htm

[Voutilainen2711]

Ville Voutilainen. *P2711R1, Making multi-param constructors of views explicit*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2711r1.html

[Voutilainen2780]

Ville Voutilainen. *P2780R0, Caller-side precondition checking, and Eval_and_throw*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2780r0.html